

#Tentang Platform & Pengalaman Pengguna

1. Aku ingin app ini bisa berjalan di semua device dan platform
2. Aku ingin pengguna itu berinteraksi nya itu secara kombinasi jadi keputusan user mau pake apa mau pake chat interface , atau mau pake input tradisional , atau mau pake voice command
3. Perihal privasi data ini aku masih bingung nih mau disimpan lokal di perangkat atau sinkronisasi cloud firebase atau enkripsi end to end , karena aku mau AI yang di dalam app ini bisa belajar nggak cuman dari data dia sendiri di perangkat masing masing user , tapi aku juga mau AI nya belajar dari device lain juga tapi pastikan privasi nya tidak tersebar Aku juga mau ada fitur supaya task ini itu gabung sama teman yang udah masuk di friend list , jadi dia bisa bikin task bareng jadi si app bisa ingetin kalau dua duanya belum ngerjain atau bahkan salah satu dari mereka udah ngerjain nih dan teman satunya malah malas

#Tentang integrasi gemini

1. peran gemini ini sebenarnya aku pengen dia all rounder ya jadi dia bisa natural language parser , smart scheduler , dan accountability coach
2. Koneksi api ini kayaknya yang aku bingung , apa kita mending train AI kita sendiri kah daripada wrapping gimana menurutmu ?

#Tentang fitur keuangan

1. kayaknya kita pakai manual input + template aja deh

#Tentang sistem perjadwalan

1. Bagaimana app tau user menunda ? itu tidak centang task tepat waktu atau user klik tunda atau deteksi aktivitas perangkat juga , lalu jika task tidak selesai user bisa pilih untuk cancel task atau reschedule , jadi minta konfirmasi user , nanti baru dikasih insight
2. Struktur data task , ya task punya prioritas , tapi durasi task nya biar jadi optional , category task tentu harus ada , dan dependensi , ada juga task repetisi harian / mingguan (mungkin dibagi jadi tab yang berbeda beda atau gimana menurutmu)

#Hasil diskusi

Dimengerti. Kita akan kunci mode ini: ****No Code, No Architecture Specs.**** Murni audit logika, ***reasoning***, dan ***brainstorming*** alur kerja (***workflow***).

Sebagai mitra diskusi, saya akan membedah ide ****Alter*** berdasarkan poin-poin yang kamu ajukan. Saya akan memberikan "Audit Logika" untuk melihat celah, tantangan, dan konsekuensi dari setiap keputusan, lalu memberikan pertanyaan pemantik untuk mempertajam konsepnya.

Mari kita bedah satu per satu.

1. Audit Interaksi: "Kombinasi Chat, Input Tradisional, & Voice"

****Logika & Reasoning:****

Keinginanmu memberikan kebebasan penuh (chat, form, voice) terdengar ideal, namun secara UX (*User Experience*) memiliki risiko ****Paradox of Choice****. Jika pengguna dihadapkan pada 3 cara input sekaligus tanpa konteks, mereka justru bisa bingung ("Saya harus ketik atau ngomong ya?").

* ****Tantangan Konsistensi Data:**** Input via chat ("Beli kopi besok") dan input via form (pilih tanggal, jam, kategori) menghasilkan struktur data yang berbeda tingkat kepastiannya. Voice dan Chat bersifat **ambigu** (perlu NLP untuk memastikannya), sedangkan Form bersifat **deterministik** (pasti).

* ****Audit Workflow:**** Bagaimana aplikasi menangani ambiguitas? Jika user bicara "Ingatkan saya rapat nanti sore" (tanpa jam jelas), apakah AI harus langsung menjadwalkan (berisiko salah) atau bertanya balik (membutuhkan interaksi lebih)?

****Pertanyaan Pemantik:****

> Daripada memberi 3 tombol paralel, apakah lebih logis jika kita menerapkan *****Adaptive Interface*****?

> * ***Default:*** Tampilan cepat (tradisional) untuk input data terstruktur.

> * ***Contextual:*** Tombol mic/chat hanya muncul dominan saat AI mendeteksi user sedang "malas mengetik" atau dalam mode "brain dumping"?

> * Bagaimana kita memastikan user tidak merasa "dipaksa" memilih metode input yang salah?

2. Audit Privasi & "AI Belajar Lintas Device"

****Logika & Reasoning:****

Ini adalah tantangan teknis dan etis terbesar. Kamu ingin AI belajar dari data global (pengguna lain) untuk menjadi lebih pintar, tapi data tidak boleh keluar dari perangkat (privasi).

* ****Konsep Federated Learning:**** Secara teori, solusinya adalah **Federated Learning**. Model AI dikirim ke HP user, belajar dari data lokal user, lalu hanya ****pola pembaruan bobot (gradients)**** yang dikirim ke server, bukan datanya.

* ****Audit Realitas:**** Implementasi Federated Learning untuk LLM (seperti Gemini) di level aplikasi individu sangat berat dan kompleks.

* ****Audit Fitur "Teman" (Collaborative Task):**** Di sini logikanya agak bertabrakan. Jika data tidak boleh tersebar, bagaimana AI bisa memberi tahu si A bahwa si B sudah mengerjakan tugas?

* ***Celah Logika:*** Untuk fitur kolaborasi, ****minimal metadata status task**** (Done/Not Done) HARUS keluar dari perangkat user ke server perantara. Ini menciptakan area abu-abu privasi.

****Pertanyaan Pemantik:****

> Untuk fitur kolaborasi, seberapa granular data yang rela kamu "korbankan" demi sinkronisasi?

> * Apakah cukup hanya status "Selesai/Belum" yang disinkronkan, sementara **catatan** atau **diskusi** dalam task tersebut tetap terenkripsi end-to-end dan tidak bisa dibaca server/AI?

> * Jika AI memberikan saran ke User A berdasarkan kebiasaan User B (yang merupakan temannya), bagaimana kita memastikan User B tidak merasa "diintip" polanya oleh User A?

3. Audit Integrasi AI: "Train Sendiri vs Wrapping Gemini"

****Logika & Reasoning:****

Kamu bertanya apakah lebih baik *train AI sendiri*. Mari kita lihat *reasoning*-nya secara *first-principles*:

* ****Opsi A: Train AI Sendiri (From Scratch/Fine-tune Open Source)****

* *Kelebihan:* Kontrol penuh, privasi data (bisa jalan lokal), tidak ada biaya API per request.

* *Kekurangan Fatal:* Biaya komputasi untuk *training* dan *inference* sangat mahal.

Model lokal (kecil) biasanya kurang cerdas dalam *reasoning* kompleks (seperti negosiasi jadwal atau motivasi psikologis). Kamu butuh tim *Data Scientist*, bukan hanya *Mobile Developer*.

* ****Opsi B: Wrapping Gemini (API)****

* *Kelebihan:* Kecerdasan tinggi "out of the box", kemampuan NLP luar biasa, hemat waktu pengembangan.

* *Kekurangan:* Biaya operasional (per token), latensi (harus online), dan data dikirim ke pihak ketiga (Google).

****Audit Keputusan:****

Mengingat Altera adalah "Otak Kedua" yang butuh kemampuan bahasa alami tinggi, ****Train Sendiri**** saat ini adalah *over-engineering* yang berisiko membuat proyek mangkrak di tengah jalan karena kompleksitas model.

****Pertanyaan Pemantik:****

> Apakah kita bisa mengambil jalan tengah?

> * Gunakan ****Gemini (Cloud)**** hanya untuk tugas berat: *Parsing bahasa alami yang rumit* dan *Strategic Planning*.

> * Gunakan ****Logika Lokal/Deterministik**** (bukan AI) untuk tugas ringan: *Notifikasi, pengecekan waktu, dan kalkulasi keuangan*.

> * Bagaimana strategimu membatasi penggunaan API Gemini agar biaya tidak membengkak saat user mulai ribuan? (Misal: *caching* respon, atau membatasi "chatting" dengan AI hanya untuk user premium?)

4. Audit Deteksi Procrastination & Rescheduling

****Logika & Reasoning:****

Kamu ingin aplikasi "menegur" dan "mereschedule", tapi juga meminta "konfirmasi user".

* **Audit Psikologis:** Ada garis tipis antara "Asisten yang Membantu" dan "Asisten yang Menghakimi". Jika aplikasi terlalu agresif menegur ("Kamu malas ya?"), user bisa merasa *annoyed* dan menghapus aplikasi.

* **Audit Workflow Reschedule:**

1. *Trigger:* Waktu task habis -> Status belum Done.
2. *Deteksi:* User membuka aplikasi lain (misal: TikTok) -> Indikasi distraksi.
3. *Intervensi:* AI menegur.
4. *Konflik:* Jika user sedang *deep work* tapi tidak mencentang task di aplikasi, apakah AI tetap menegur? (False Positive).

Pertanyaan Pemantik:

- > Bagaimana kita mendefinisikan "Procrastination" secara logis agar tidak salah tuduh?
- > * Apakah cukup dengan *timestamp* (tidak dicentang saat deadline)?
- > * Atau perlu *behavioral signal* (user membuka HP tapi tidak membuka Altera)?
- > * Saat AI menawarkan reschedule, apakah dia hanya memindahkan jamnya, atau dia juga harus "meminta izin" untuk memindahkan task lain yang bentrok? (Ini menyangkut logika *Constraint Satisfaction Problem*).

5. Audit Struktur Data Task (Hierarki & Dependensi)

Logika & Reasoning:

Kamu menyebutkan: Prioritas, Optional Duration, Category, Dependencies, Repetition.

* **Audit Dependensi (Ketergantungan):** Ini adalah fitur yang *powerful* tapi sangat rumit di level UI/UX. Jika Task A (Dependen) belum selesai, Task B tidak bisa mulai.

* *Risiko:* Jika user lupa mengerjakan Task A, seluruh rantai task di belakangnya bisa berantakan (domino effect). Bagaimana AI menangani ini? Apakah dia otomatis memajukan semua task di belakangnya?

* **Audit Repetisi vs Dependencies:** Bagaimana logika task berulang yang punya dependensi? (Contoh: "Cuci baju" (berulang) harus selesai sebelum "Setrika baju" (berulang)). Jika minggu ini user skip "Cuci baju", apakah "Setrika baju" ikut hilang atau tetap ada?

Pertanyaan Pemantik:

- > Untuk menghindari kompleksitas berlebihan di awal, apakah kita perlu membatasi "Depth of Dependency"?
- > * Misalnya: Task hanya boleh bergantung pada 1 task sebelumnya (Linear), tidak boleh bercabang (Tree/Graph)?
- > * Bagaimana kita memvisualisasikan dependensi ini di layar HP yang kecil agar tidak membingungkan user?

6. Audit Fitur Keuangan (Manual Input + Template)

****Logika & Reasoning:****

Keputusan menggunakan *Manual Input + Template* adalah keputusan paling rasional untuk MVP (*Minimum Viable Product*).

* ****Audit Konsistensi:**** Tantangan fitur keuangan manual adalah *user retention*. User biasanya rajin di minggu pertama, lalu malas input di minggu kedua.

* ****Peran AI di sini:**** Di sinilah "Wrapping Gemini" bisa bersinar. User tidak perlu memilih kategori dari dropdown. User cukup ketik/ngomong: "Beli bensin 20rb, kopi 15rb, parkir 2rb". AI harus cukup cerdas untuk memecah satu kalimat itu menjadi 3 entri transaksi terpisah dengan kategori yang tepat.

****Pertanyaan Pemantik:****

> Bagaimana kita membuat proses input manual ini "semenyamankan" mungkin?

> * Apakah kita butuh fitur "Quick Capture" di mana user hanya perlu ketik angka dan AI menebak sisanya berdasarkan konteks lokasi/waktu?

> * Bagaimana logika validasinya jika AI salah menebak kategori? (Misal: "Beli pulsa" dikira "Pulsa Kerja" padahal "Pulsa Pribadi").

Kesimpulan Audit Sementara

Secara logika, ****Altera**** memiliki potensi besar, namun tantangan utamanya bukan pada "bisa tidaknya dibuat", melainkan pada ****Manajemen Kompleksitas****:

1. ****Kompleksitas Privasi vs Kolaborasi:**** Kamu harus memutuskan batasan data mana yang *lokal murni* dan mana yang *wajib sinkron* untuk fitur teman.
2. ****Kompleksitas AI vs Cost:**** Menggunakan Gemini untuk *semua* interaksi akan mahal. Kamu butuh strategi "Hybrid Intelligence" (AI hanya untuk tugas kognitif tinggi, kode biasa untuk logika rutin).
3. ****Kompleksitas UX:**** Memberi terlalu banyak kebebasan (input method, fitur task rumit) bisa membuat aplikasi terasa berat.

****Langkah Brainstorming Selanjutnya:****

Dari 6 poin audit di atas, mana yang menurutmu paling "berisiko" atau paling membingungkan logikanya saat ini? Kita bisa bedah lebih dalam satu poin tersebut sebelum melangkah ke poin lainnya.